

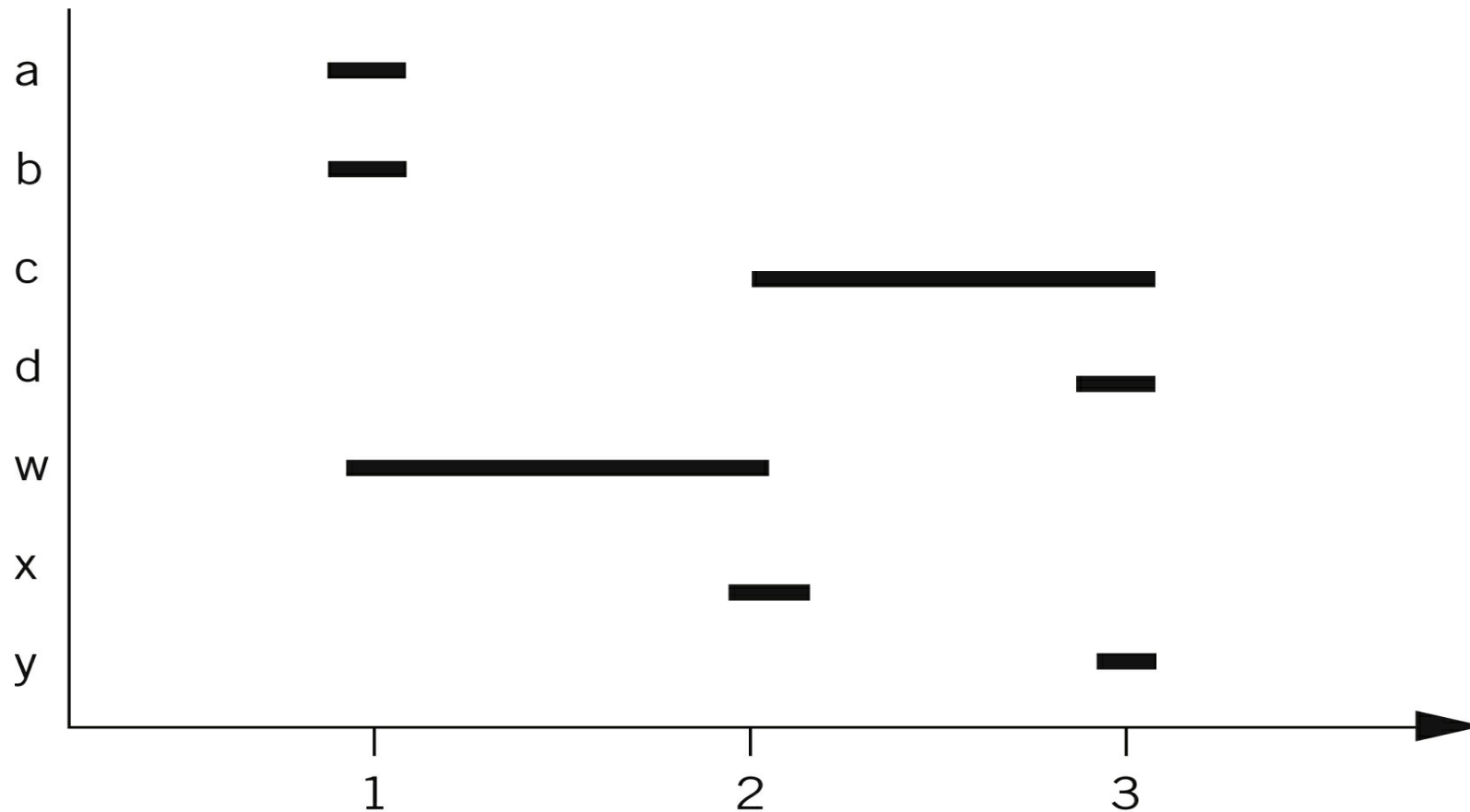
REGISTER & OPERATOR ALLOCATION

Register allocation is a very important compilation phase. Given a block of code, choose assignments of variables (both declared and temporary) to registers to minimize the total number of required registers

PROBLEM 1

```
w = a + b; /* statement 1 */  
x = c + w; /* statement 2 */  
y = c + d; /* statement 3 */
```

LIFE TIME GRAPH



lifetime graph that shows the statements on which each statement is used. Here is a lifetime graph in which the x axis is the statement number in the C code and the y axis shows the variables.

Here is the ARM assembly code that uses the above register assignment:

LDR r0,[p_a] ; load a into r0 using pointer to a (p_a)

LDR r1,[p_b] ; load b into r1

ADD r3,r0,r1 ; compute a + b

STR r3,[p_w] ; w = a + b

LDR r2,[p_c] ; load c into r2

ADD r0,r2,r3 ; compute c + w, reusing r0 for x

STR r0,[p_x] ; x = c + w

LDR r0,[p_d] ; load d into r0

ADD r3,r2,r0 ; compute c + d, reusing r3 for y

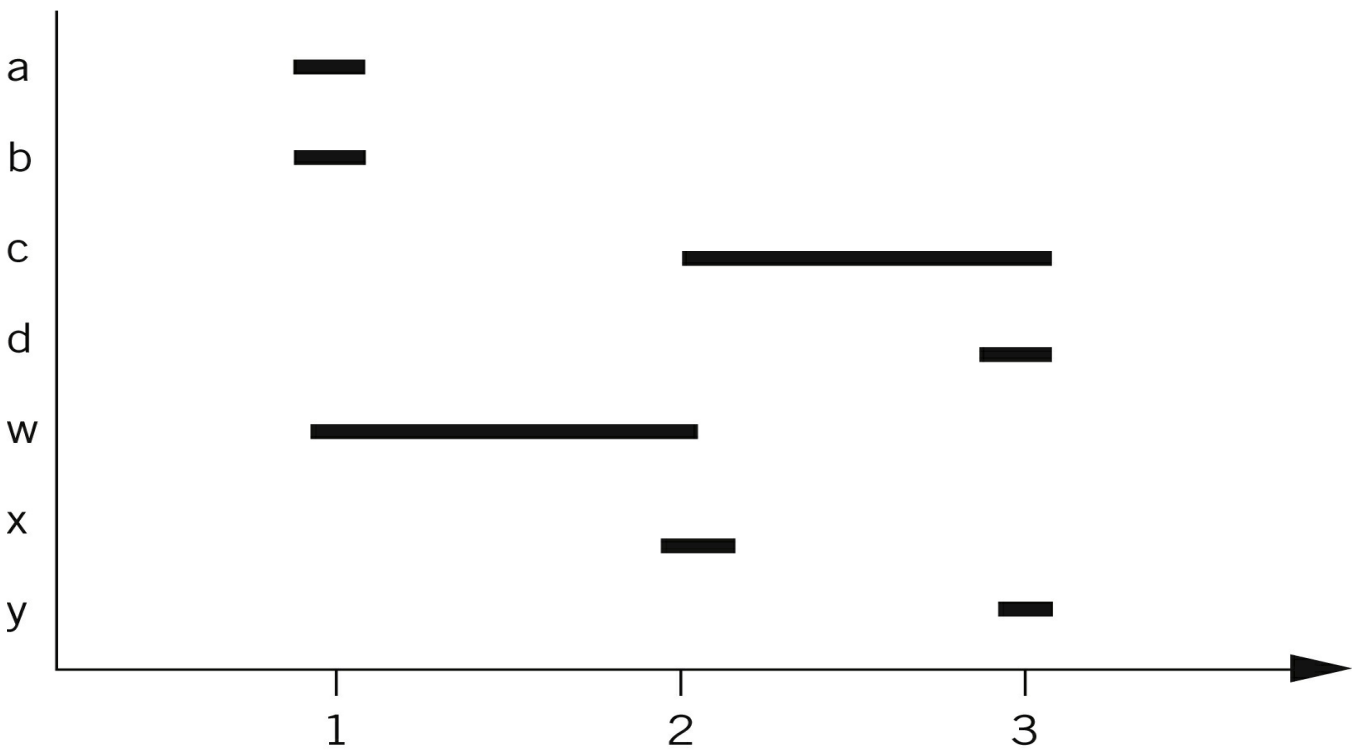
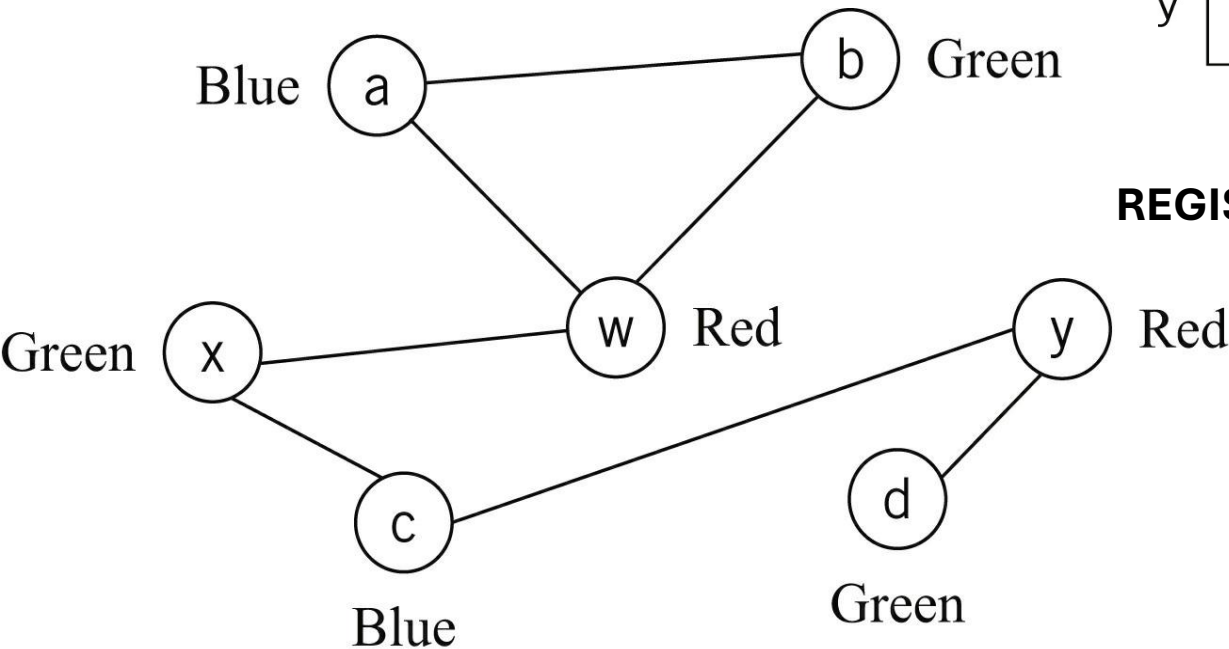
STR r3,[p_y] ; y = c + d

If a section of code requires more registers than are available, we must spill some of the values out to memory temporarily. After computing some values, we write the values to temporary memory locations, reuse those registers in other computations, and then reread the old values from the temporary locations to resume work. Spilling registers is problematic in several respects: it requires extra CPU time and uses up both instruction and data memory. Putting effort into register allocation to avoid unnecessary register spills is worth your time.

PROBLEM 1

INTERFERENCE GRAPH

Conflict graph and solving a graph coloring problem



REGISTER ALLOCATION

a r0
b r1
c r0
d r1
w r2
x r1
y r2

PROBLEM 1

w = a + b; /* statement 1 */
x = c + w; /* statement 2 */
y = c + d; /* statement 3 */

AFTER ALLOCATION

r2=r0+r1
r1=r0+r2
r2=r0+r1

Operator Scheduling for Register Allocation

PROBLEM 2

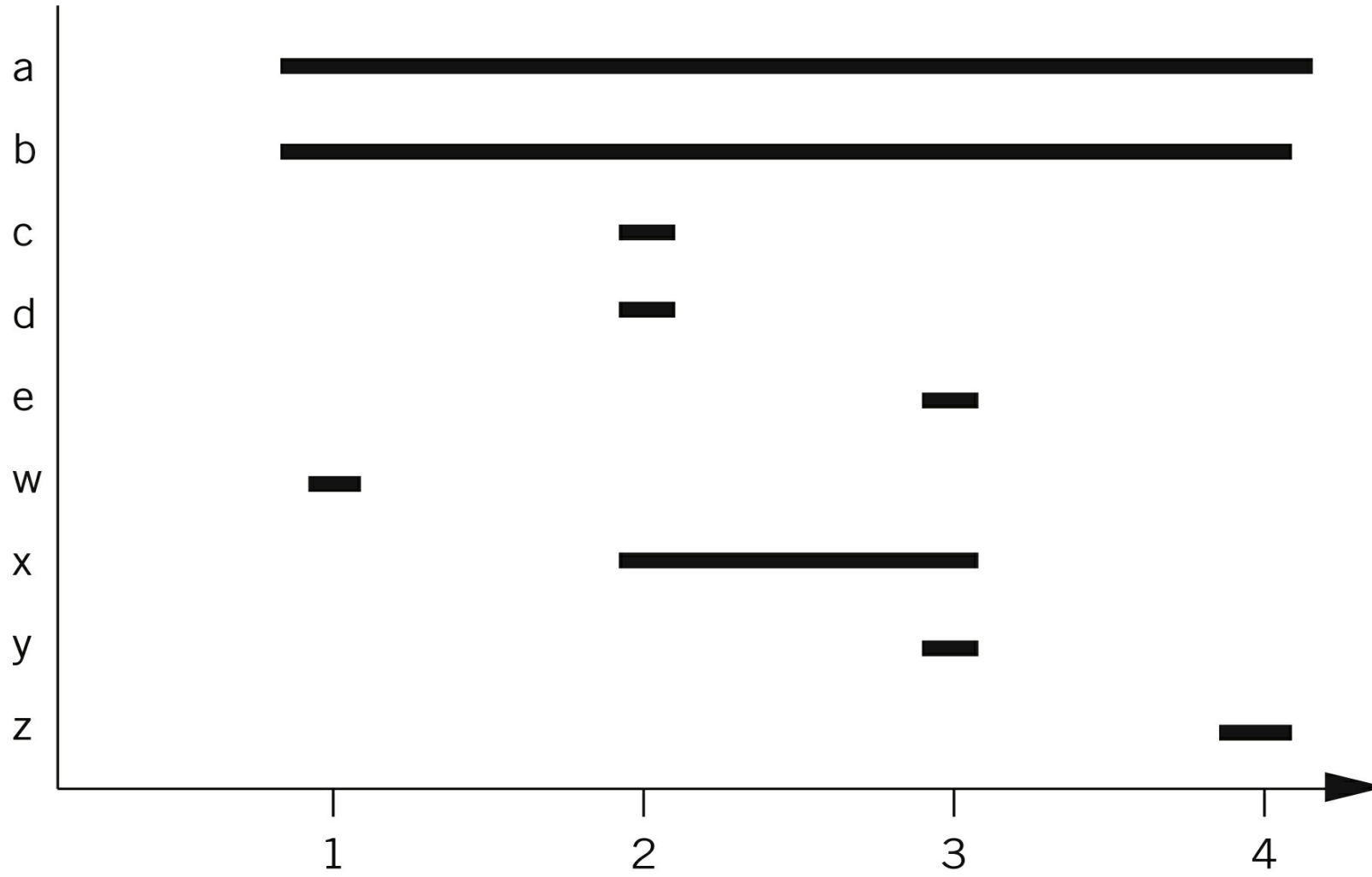
`w = a + b; /*statement 1 */`

`x = c + d; /*statement 2 */`

`y = x + e; /*statement 3 */`

`z = a - b; /*statement 4 */`

LIFE TIME GRAPH



AFTER OPTIMISATION-LIFE TIME GRAPH

```
w = a + b; /* statement 1 */
z = a - b; /* statement 29 */
x = c + d; /* statement 39 */
y = x + e; /* statement 49 */
```

